

π_0 · arXiv 2410.24164 · RSS 2025

Empiricist

11+3 · seat 5

H1-H3 / H5 · H / NFE / FT grid
1-4 GPU-days on public OpenPI

Sep 28 paper club · VLA Architectures

Spine: PaliGemma + OT-CFM action chunks (DDPM→flow)

Physical Intelligence · H=50 · ~10 Euler · ~73 ms RTX4090 · NOT a classical planner

Hypotheses · H1-H3 / H5

H1 horizon

Mid commit ($\sim 0.5\text{--}1\text{ s}$ / $H \approx 16\text{--}32$)
beats too-short and full open-loop $H=50$

H2 NFE

Success saturates $\sim 5\text{--}10$ Euler steps;
1-2 hurt; >10 buys little latency

H3 fine-tune

FT from $\pi \gg$ zero-shot / scratch
on multi-stage tasks (up to $\sim 2\times$)

H5 FM vs DP

FM reaches target at lower NFE / ms
than DP at matched backbone

P0 grid · H × NFE × FT

Public OpenPI ckpts · schematic expected success (not measured)

H \ NFE	1	2	4	5	10	20
H=8	0.29	0.33	0.37	0.38	0.41	0.42
H=16	0.41	0.53	0.69	0.74	0.86	0.86
H=25	0.41	0.53	0.69	0.74	0.86	0.86
H=50	0.29	0.33	0.37	0.38	0.41	0.42

Init arm

zero-shot
FT from π
scratch

Expect

peak mid-H
NFE ~5-10
FT $\gg$ scratch

Falsifiers: H=50 always best · NFE=1=NFE=10 · scratch $\geq$ FT · DP>FM @ equal ms

Experiment priority

P0

$H \in \{8,16,25,50\} \cdot NFE \in \{1,2,4,5,10,20\} \cdot FT$ vs zero-shot/scratch

P1

Cross-embodiment · FM vs DP head (matched backbone)

P2

τ -schedule · open-loop commit · VLM init vs scratch expert

Defer

Full $\sim 10k$ -h pretrain · full laundry/box suite (PI hardware)

Budget honesty: P0 on public OpenPI = 1-4× GPU days (not weeks)

GPU honesty

NFE / horizon eval (frozen)	1× GPU	hours	12-24 GB
LoRA / partial FT π_0 - $\pi_{0.5}$	1× GPU	1-4 days	24-48 GB
Action-FM toy / Push-T	1× GPU	hours-1 day	4-10 GB
Paper-scale 700k @ 3.3B	multi-node	weeks	OUT OF SCOPE

Always log (H, K, Hz, NFE, ms, GPU name, max_memory_allocated)

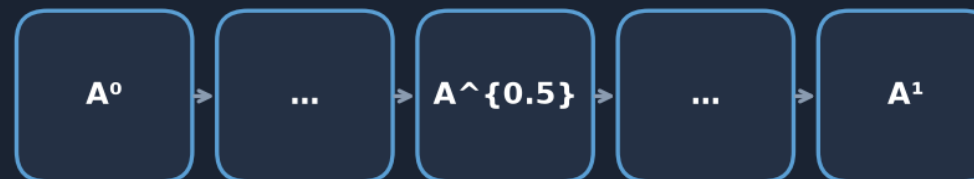
OT-CFM action chunks · DDPM → flow

Linear path $A^\tau = \tau A + (1-\tau)\varepsilon$ · target $u = A - \varepsilon$ · $\tau \sim \text{shifted Beta}(1.5, 1)$

Straight OT path in action space



Inference: ~10 Euler steps



$$A^{\{\tau+\delta\}} = A^\tau + \delta \cdot v_\theta(A^\tau, o) \quad \cdot \quad \delta=0.1$$

Chunk → commit → replan (DP/ACT pattern; FM head)

Train H=50

propose full chunk

Execute K

16@20Hz / 25@50Hz

Open-loop

no temporal ensemble

Replan

new infer; RTC later

Orthogonal control knobs

First-class for 2026 · not just the VLM backbone

H

train horizon

paper default 50
sweep {8,16,25,50}

T_a / K

open-loop commit

16@20Hz · 25@50Hz
mid ≈ 0.5 –1 s sweet

NFE

Euler steps

paper ~ 10
saturates ~ 5 –10

async

overlap / RTC

soft-inpaint
exec26 / keep4

Σ

noise corr.

isotropic $\rightarrow$
 $0.5\Sigma + 0.5I$

Latency budget (Table I · RTX 4090 · 3 cams)

Image enc

14 ms

Obs forward

32 ms

$\times 10$ FM

27 ms

On-board

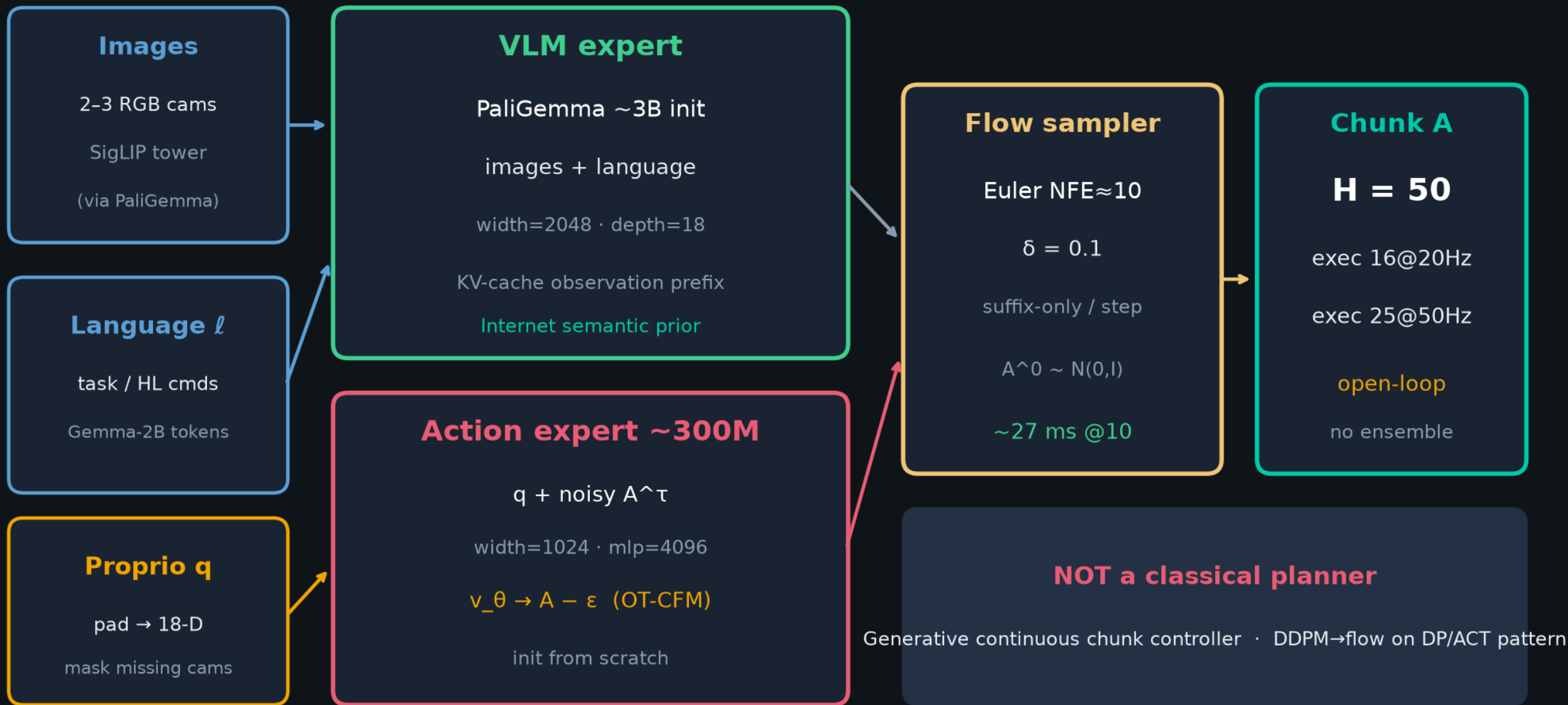
73 ms

+Wi-Fi

~ 86 ms

Architecture · PaliGemma + OT-CFM action expert

~3.3B total · MoE dual-stream (Transfusion-inspired) · blockwise causal attention



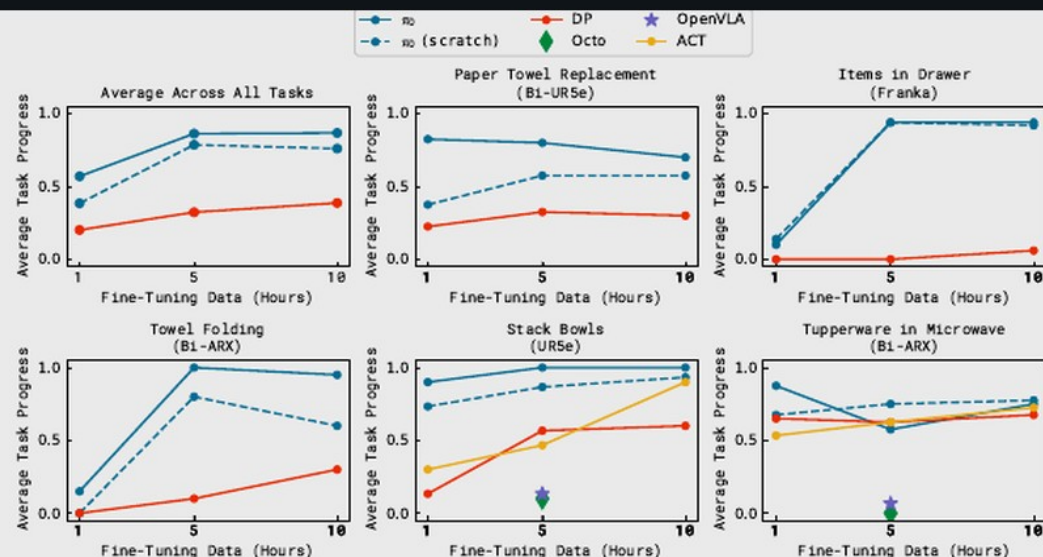


Fig. 11: **Fine-tuning with varying amounts of data.** π_0 can learn some easier tasks even with smaller amounts of data, and the pre-trained model often attains a larger improvement over the model trained from scratch.

that are more similar to the pre-training data, though the pre-trained model is frequently better than the non-pre-trained model, sometimes by as much as 2x.

D. Mastering complex multi-stage tasks

In our final set of experiments, we tackle a range of challenging multi-stage tasks via a combination of fine-tuning and language. For some of these tasks, data is present in pre-training, but fine-tuning is required to attain mastery. For some, no data is present in pre-training. The tasks in this evaluation, shown in Figure 12, are:

Laundry folding: This task requires a static (non-mobile) bi-manual system to fold articles of clothing. The clothing items start in a randomized crumpled state in a bin, and the goal is to take out the item, fold it, and place it on top of a stack of previously folded items. The randomized initial configuration of the crumpled laundry presents a major challenge, since the policy needs to generalize to any configuration. This task is present in pre-training.

Mobile laundry: Here, the Fibocom mobile robot in Figure 5 has to fold laundry, facing many of the same challenges while

of varying shapes and sizes, and perform complex dexterous motions, such as twisting the gripper to pick up large plates and carefully grasping thin, delicate items such as glasses. The robot must handle dense clutter and intelligently sequence various behaviors — for example, to clean off a plate with trash, it must first pick up the plate, then shake its contents into the garbage, and then place the plate in the bin. This task is not present in pre-training.

Box building: The robot has to assemble a cardboard box that starts in a flattened state. This task presents a number of major challenges: the box needs to be bent in the right way, and the robot needs to hold down parts of the box while folding others, utilizing both arms and even the surface of the table to brace during folding motions. The robot might need to retry some folds, requiring a reactive and intelligent strategy. This task is not present in pre-training.

To-go box: This task requires moving several food items from a plate into a to-go box, requiring packing the items into the box so that they do not stick out, and then closing the box with both arms. This task is not present in pre-training.

Zhiyuan Xu, Ning Liu, Ran Cheng, Chaomin Shen, Yaxin Peng, Feifei Feng, et al. Scaling diffusion policy in transformer to 1 billion parameters for robotic manipulation. *arXiv preprint arXiv:2409.14411*, 2024.

APPENDIX

A. Contributions

The authors contributed to the following areas (listed alphabetically):

Data and operations: Noah Brown, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Liyiming Ke, Suraj Nair, Lucy Shi, and Anna Walling.

Evaluation experiments: Kevin Black, Michael Equi, Chelsea Finn, Brian Ichter, Liyiming Ke, Adrian Li-Bell, Suraj Nair, Karl Pertsch, and Lucy Shi.

Model design: Kevin Black, Brian Ichter, Sergey Levine, Karl Pertsch, Lucy Shi, and Quan Vuong.

Post-training: Michael Equi, Chelsea Finn, Liyiming Ke, Adrian Li-Bell, Suraj Nair, and Lucy Shi.

Pre-training: Kevin Black, Danny Driess, Brian Ichter, Sergey Levine, Karl Pertsch, Lucy Shi, and Quan Vuong.

Robot hardware: Noah Brown, Adnan Esmail, Chelsea Finn, Tim Jones, and Mohith Mothukuri.

Robot software: Karol Hausman, Szymon Jakubczak, Sergey Levine, James Tanner, and Haohuan Wang.

Training infrastructure: Kevin Black, Michael Equi, Sergey Levine, Adrian Li-Bell, Suraj Nair, Quan Vuong, Haohuan Wang, and Ury Zhilinsky.

Writing and illustration: Kevin Black, Chelsea Finn, Lachy Groom, Karol Hausman, Brian Ichter, Sergey Levine, and Quan Vuong.

B. Model Architecture Details

transformer is $W_3 \cdot \text{swish}(W_2 \cdot \text{concat}(W_1 \cdot \mathbf{a}_t^\tau, \phi(\tau)))$, where $\phi: \mathbb{R} \rightarrow \mathbb{R}^w$ is a sinusoidal positional encoding function [51], $W_1 \in \mathbb{R}^{w \times d}$, $W_2 \in \mathbb{R}^{w \times 2w}$, $W_3 \in \mathbb{R}^{w \times w}$, d is the action dimension, and w is the embedding dimension (or *width*) of the action expert.

Attention mask. π_0 uses a blockwise causal attention mask with 3 blocks: $[\mathbf{I}_t^1, \dots, \mathbf{I}_t^n, \ell_t]$, $[\mathbf{q}_t]$, and $[\mathbf{a}_t^\tau, \dots, \mathbf{a}_{t+H-1}^\tau]$. Within each block, there is full bidirectional attention, whereas the tokens in each block cannot attend to the tokens in future blocks. The first block includes the input modalities from PaliGemma’s VLM pre-training, which are prevented from attending to future blocks (which include new inputs) to minimize distribution shift from said pre-training. The robot state $\mathbf{q}_t$ is its own block because it does not change with each flow matching integration step; preventing it from attending to the final block allows its corresponding keys and values to be cached during sampling. The final block corresponds to the noisy actions $\mathbf{A}_t^\tau$, which can attend to the full input sequence.

Action expert. π_0 is implemented as a single transformer with two sets of weights (also known as experts [45]), where each token is routed to one of the experts; the weights interact only through the transformer’s self-attention layers. The images and language prompt, $[\mathbf{I}_t^1, \dots, \mathbf{I}_t^n, \ell_t]$, are routed to the larger VLM backbone, which we initialize from PaliGemma. The inputs not seen during VLM pre-training, $[\mathbf{q}_t, \mathbf{A}_t^\tau]$, are routed to the action expert. PaliGemma is based on the Gemma 2B [49] language model, which uses multi-query attention [44] and a configuration of $\{\text{width}=2048, \text{depth}=18, \text{mlp_dim}=16,384, \text{num_heads}=18, \text{num_kv_heads}=1, \text{head_dim}=256\}$. Since the experts interact only in the self-attention layers, *width* and *mlp_dim* do not necessarily need to match between experts. To speed up inference (which requires multiple forward passes of the

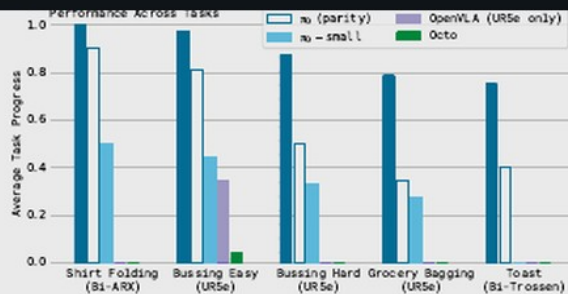


Fig. 7: **Out-of-box evaluation results:** We evaluate π_0 trained for the full 700k steps, a version trained for 160k steps that matches the number of updates for baseline models, π_0 -small, and three baselines: OpenVLA and Octo trained on all of our data, and OpenVLA trained only on the UR5e tasks (which we found to work better on UR5e tasks). Across all tasks and all comparisons, even the “parity” version of our model outperforms all baselines, and the full version of our model achieves the best results by a large margin.

below the performance of π_0 . Octo does support action chunks, but has a comparatively limited representational capacity. This comparison illustrates the importance of combining large, expressive architectures with the ability to model complex distributions via flow matching or diffusion. Additionally, the comparison to π_0 -small illustrates the importance of incorporating VLM pre-training. Unfortunately, it is hard to make this last comparison fair: π_0 -small uses fewer parameters, but larger models are difficult to use without pre-training. Overall, these experiments show that π_0 provides a powerful pre-trained model with the ability to effectively perform a variety of tasks with a variety of robots, with much better performance than prior models.

B. Following language commands

In the next set of experiments, we fine-tune the base π_0 model to follow language commands in a set of evaluation domains. We compare this fine-tuned π_0 model with the π_0 -small model described in Section IV, which we found to be the strongest baseline in the previous section. Recall that π_0 -small does *not* use a VLM initialization. This experiment therefore aims to measure how much VLM pre-training boosts our model’s ability to follow language instructions.



Fig. 8: **The tasks in our language evaluation.** We evaluate our model on 3 different language-conditioned tasks, each of which requires following a sequence of intermediate language commands. The tasks involve bussing a table (top) to put dishes in a bin and garbage in a trash bin, setting a table (middle) by taking items out of a bin, and packing a shopping bag (bottom).

task consists of numerous such segments. The tasks in this evaluation consist of:

Bussing: the robot must clean a table, placing dishes and cutlery in a bin, and trash into a trash bin.

Table setting: the robot must take out items from a bin to set a table, including a place mat, dishes, silverware, napkin, and cups, and adjust them according to language instructions.

Grocery bagging: the robot must pack grocery items, such as bags of coffee beans, barley, marshmallow, seaweed, almonds, spaghetti, and cans into a bag.

In Figure 8, we show the language-conditioned tasks in our evaluation and present the evaluation results. We evaluate five different conditions. π_0 -flat (and π_0 -small-flat) corresponds to directly command the model with the task description (e.g., “bag the groceries”), without intermediate language commands. π_0 -human (and π_0 -small-human) provides intermediate step commands (e.g., which object to pick and where to place it) from an expert human user. These conditions evaluate each model’s ability to follow more detailed language commands: while these intermediate commands provide considerable information for how to perform the task, the model must be able to understand and follow those commands to benefit from them. Finally, π_0 -HL evaluates π_0 with high-level commands provided by a high-level VLM, as discussed in Section V-B. This condition is also autonomous, without any human expert.

The results in Figure 9, averaging over 10 trials per task, show that the language following accuracy of π_0 is significantly better than that of π_0 -small. This suggests a significant improvement from the larger pre-trained VLM initialization. This capability translates to an improvement

Empiricist · close

P0: $H \times NFE \times FT$ on public OpenPI. 1-4 GPU-days — not weeks.

Quote success only with (H, K, Hz, NFE, ms, GPU)

Q&A · Sep 28 · π_0 / 2410.24164

π_0 : A Vision-Language-Action Flow Model for General Robot Control

Physical Intelligence

Kevin Black, Noah Brown, Danny Driess, Adnan Esmail, Michael Equi, Chelsea Finn, Niccolo Fusai, Lachy Groom, Karol Hausman, Brian Ichter, Szymon Jakubczak, Tim Jones, Liyiming Ke, Sergey Levine, Adrian Li Bell, Mohith Mohukuri, Suraj Nair, Karl Pertsch, Lucy Xiaoqiang Shi, James Tapper, Quan Vuong